

Datenexploration

April 28, 2026

I758 Wissens- und KI-basierte Systeme

1 Explorative Datenanalyse: Verteilungen und Zusammenhänge

(c) Ricardo Knauer, Raphael Wallsberger, Christina Kratsch

Willkommen bei der ersten praktischen Übung zum Thema explorative Datenanalyse! Wir haben uns während der Vorlesung bereits mit einigen Verfahren beschäftigt, mit denen man einen Datensatz erkunden kann. Im Folgenden wollen wir diese Verfahren auch praktisch anwenden. Zum Glück ist die explorative Analyse von Daten so weit verbreitet, dass es bereits eine Vielzahl an Bibliotheken für genau diesen Zweck gibt. Wir werden für unsere Analyse eine der bekanntesten Bibliotheken verwenden, um interaktive Diagramme zu erstellen: Plotly. Plotly verfügt über Schnittstellen zu einer ganzen Reihe an Programmiersprachen, darunter auch [Python](#). Mit der ersten praktischen Übung verfolgen wir folgende Lernziele:

- Sie sollen lernen, wie Sie Daten mithilfe von Säulen- oder Balkendiagrammen, Histogrammen, Streudiagrammen und Heatmaps in Plotly visualisieren können.
- Sie sollen den Wert von Funktionsargumenten zur Visualisierung sach- und zweckorientiert anpassen können.
- Sie sollen lernen, wie man eigene Hypothesen über einen Datensatz aufstellen und diese im Rahmen einer explorativen Analyse testen kann.

Voraussichtlich werden Sie für diese Übung maximal 2h benötigen.

1.1 Der Datensatz

```
[1]: # Setup: installieren Sie die benötigten Bibliotheken  
  
# conda activate wikisys  
# conda install pandas matplotlib plotly scipy
```

Für die explorative Analyse von Daten braucht man als Ausgangspunkt erst einmal eins: Daten. Für unsere praktischen Übungen werden wir einen Datensatz verwenden, der uns freundlicherweise von [Prof. Dr. Stephan Matzka](#) zur Verfügung gestellt wurde. Der Datensatz befasst sich mit einer Fertigungszelle, in der drei Maschinen Produkte herstellen. Unser Datensatz `machine_data.csv` liegt im sogenannten csv-Format vor (englisch für *comma-separated values*). Das ist ein gängiges Format zum Speichern strukturierter Daten, bei dem einzelne Werte durch Kommas oder andere Trennzeichen separiert sind. Wenn wir uns die Datei anschauen, erkennen wir, dass die Trennzeichen in unserem Fall Semikolons sind und dass Floats ein Komma anstelle eines Punktes als Dezimaltrennzeichen aufweisen. Um den Datensatz einzulesen, nutzen wir die [pandas](#)-Bibliothek -

die erste Wahl bei der Arbeit mit tabellarischen Daten in Python. Eine kleine Anmerkung: Unter diesem Textabschnitt finden Sie einen Codeblock. Codeblöcke wie diesen können Sie ausführen, indem Sie auf den Block klicken und danach gleichzeitig SHIFT und ENTER auf der Tastatur drücken. Die entsprechende Ausgabe erscheint gegebenenfalls unter dem Block. Codeblöcke, die Sie zwischen den Texten in dieser Übung sehen, sollten Sie in der vorgegebenen Reihenfolge ausführen. Bei leeren Codeblöcken müssen Sie zuvor noch Ihren eigenen Code ergänzen.

```
[2]: import pandas as pd

df = pd.read_csv("data/machine_data.csv", delimiter=";", decimal=",")
```

Sie sehen, dass wir zuerst die pandas-Bibliothek importieren. Um bei Funktionsaufrufen aus dieser Bibliothek nicht jedes Mal `pandas` ausschreiben zu müssen, lassen wir Python wissen, dass wir das Kürzel `pd` anstelle von `pandas` benutzen möchten. Unseren csv-Datensatz können wir einlesen, indem wir einfach den Befehl `pd.read_csv()` ausführen. Dabei müssen wir der Funktion natürlich mitteilen, wo sich unsere Daten befinden und dass wir `;` als Trennzeichen sowie `,` als Dezimaltrennzeichen verwenden möchten. Der Datensatz wird abschließend in der Variable `df` in Form einer Tabelle gespeichert. Schauen wir uns die ersten paar Zeilen dieser Tabelle an. Dazu benutzen wir pandas' `head()`-Funktion:

```
[3]: df.head(10)
```

```
[3]:  Maschine  Mode Produkt  Strom / A  Drehmoment / Nm  Drehzahl / 1/min  \
0         A    2.0      Y    22.782      51.79      1461
1         A    1.0      Y    19.793      45.03      1460
2         C    3.0      X    25.581      52.00      1463
3         C    3.0      Y    28.145      57.30      1461
4         C    2.0      Y    25.544      51.84      1466
5         A    2.0      X    21.426      48.64      1463
6         B    2.0      Y    22.786      50.92      1462
7         C    2.0      X    23.021      46.85      1462
8         B    2.0      Y    22.255      49.69      1463
9         C    3.0      X    26.195      53.29      1462
```

```
Temp Umgebung / degC  Temp Umrichter / degC  Temp Werkzeug / degC  \
0                    18.7                23.1                98.7
1                    21.0                21.5                87.5
2                    19.9                21.8                98.1
3                    20.7                25.4               109.7
4                    19.0                24.1               100.8
5                    21.5                23.3                95.1
6                    20.9                22.8                98.2
7                    21.3                23.8                93.1
8                    21.5                24.4                97.7
9                    21.9                25.3               104.1
```

```
Bearbeitungszeit / s
0                    23.8
```

1	25.3
2	19.9
3	22.3
4	23.8
5	21.4
6	24.8
7	21.4
8	24.8
9	19.9

Die ersten drei Merkmale scheinen in Kategorien eingeteilt zu sein, bei den anderen Merkmalen handelt es sich anscheinend um Floats oder Integers. **Maschine** und **Produkt** sind offensichtlich nominale Merkmale; **Mode** ist hingegen ein ordinales Merkmal, da Energiesparbetrieb (1.0), Normalbetrieb (2.0) und Hochleistungsbetrieb (3.0) eine Rangordnung aufweisen. pandas verfügt natürlich nicht über diese Information aus der Dokumentation, daher müssen wir den ersten drei Merkmalen noch den passenden Datentyp zuweisen. Hierzu nutzen wir pandas' `Categorical()`-Funktion:

```
[4]: df["Maschine"] = pd.Categorical(df["Maschine"])
     df["Produkt"] = pd.Categorical(df["Produkt"])
     df["Mode"] = pd.Categorical(df["Mode"], ordered=True)
```

Wie man in dem oberen Codeblock erkennen kann, kann man auf Spalten der Tabelle zugreifen, indem man den Namen der Spalte (als String) zwischen zwei eckige Klammern schreibt. Um pandas mitzuteilen, ob es sich bei einem Merkmal um ein nominales oder um ein ordinales Merkmal handelt, benutzt man in der `Categorical()`-Funktion das Argument `ordered`. Das Argument ist standardmäßig auf `False` gesetzt. Bei ordinalen Merkmalen muss man der Funktion daher explizit den Wert `True` mitgeben. Nachdem wir unseren Merkmalen in der Tabelle also nun die richtigen Datentypen zugewiesen haben, können wir uns endlich den Visualisierungen zuwenden.

Ein Merkmal, ein Diagramm, viele Kenngrößen

Sie haben in der Vorlesung bereits erfahren, dass es bei der explorativen Analyse zuerst meist sinnvoll ist, sich anzuschauen, wie oft die verschiedenen Werte in jedem Merkmal vorkommen. Bei nominalen und ordinalen Merkmalen bieten sich Säulen- oder Balkendiagramme zur Visualisierung an, bei metrischen Merkmalen Histogramme. Um interaktive Diagramme zu erstellen, können wir pandas ganz einfach mit Plotly kombinieren. Dazu setzen wir das sogenannte Backend zur Darstellung von Diagrammen auf "plotly":

```
[5]: pd.options.plotting.backend = "plotly"
```

Ein [Säulendiagramm](#) können wir darstellen, indem wir die Anzahl der Werte pro Kategorie mit der `value_counts()`-Funktion zählen und danach `plot(kind="bar")` aufrufen. Für das nominale Merkmal **Maschine** sähe das wie folgt aus:

```
[6]: df["Maschine"].value_counts().plot(kind="bar")
```

Das entsprechende [Balkendiagramm](#) sähe so aus:

```
[7]: df["Maschine"].value_counts().plot(kind="bar", orientation="h")
```

AUFGABE 1: Stellen Sie selbstständig ein Säulen- oder Balkendiagramm für das nominale Merkmal **Produkt** dar. Was fällt Ihnen auf?

```
[8]: df["Produkt"].value_counts().plot(kind="bar")
```

Geben Sie nachfolgend Ihre schriftliche Antwort zur AUFGABE 1:

Am häufigsten wird Produkt “Y” bearbeitet (541 mal), dicht gefolgt von “X” (458 mal).

Auffällig ist, dass nur ein einziges mal das Produkt “Z” bearbeitet wurde.

Widmen wir uns als nächstes den metrischen Merkmalen. Metrische Merkmale kann man in Wertebereiche (englisch *bins*) einteilen und in einem [Histogramm](#) visualisieren. Ein Histogramm stellt eine Häufigkeitsverteilung dar, die man mit einigen Eigenschaften charakterisieren kann. Eventuell möchten Sie auf Wikipedia kurz nachschauen, was ein Mittelwert, eine Standardabweichung, ein Median sowie Quantile sind.

Ein Histogramm lässt sich mit Plotly ganz einfach darstellen, zum Beispiel für die Drehzahl pro Minute:

```
[9]: df["Drehzahl / 1/min"].plot(kind="histogram")
```

AUFGABE 2: Versuchen Sie, mit der Histogramm-Funktion zu spielen: stellen Sie selbstständig ein Histogramm von der Stromaufnahme in Ampere dar. Was fällt Ihnen an dieser Verteilung auf?

```
[10]: df["Strom / A"].plot(kind="histogram", nbins=100)
```

Geben Sie nachfolgend Ihre schriftliche Antwort zur AUFGABE 2:

Das Diagramm zeigt die Verteilung der Stromnutzung bei einzelnen Fertigungsschritten. Es sind 5 glockenförmige Häufungen zu erkennen (ähnlich zu Normalverteilungen), wobei die linken beiden (ca. bei 20 & 21) sehr nah und fast überlappend liegen und die äußerste (bei 28) nur sehr klein ist.

Die Häufungen liegen bei:

1. 20
2. 21
3. 23
4. 26
5. 28

Die Unterschiede kommen vermutlich durch die verschiedenen Maschinen und deren Operating-Modes zustande.

Vielleicht ist Ihnen aufgefallen, dass sich die Anzahl der Wertebereiche zwischen den beiden Histogrammen unterscheidet. Im Histogramm zur Drehzahl pro Minute gibt es 12 solcher Bereiche, im Histogramm zur Stromaufnahme in Ampere 24. In Plotly gibt es einen Algorithmus, der die optimale Anzahl und Breite der Wertebereiche automatisch festlegt. Man kann allerdings über das `nbins`-Argument zumindest die maximale Anzahl der Wertebereiche bestimmen.

AUFGABE 3: Setzen Sie das `nbins`-Argument auf 100 und beobachten Sie, wie sich das Histogramm verändert. Fällt Ihnen bei der Häufigkeitsverteilung etwas auf, das Ihnen bei einer kleineren Anzahl von Wertebereichen vielleicht nicht aufgefallen wäre (Stichwort: Ausreißer)? Lohnt sich das Testen einer größeren Anzahl von Wertebereichen oder genügt die Standardeinstellung?

```
[11]: df["Strom / A"].plot(kind="histogram", nbins=100)
```

Geben Sie nachfolgend Ihre schriftliche Antwort zur AUFGABE 3:

Durch die erhöhte Auflösung lassen sich auch sehr eng gruppierte Häufungen, also welche, bei denen die Werte sehr dicht beieinander liegen, erkennen. So wird beispielsweise auch die Grenze zwischen den ersten beiden Häufungen sowie die letzte Häufung klarer erkennbar.

Nachteil ist allerdings, dass eine zu hohe Anzahl an *bins* das Erkennen von Häufungen schwerer macht, da mehr Lücken in den Werten sichtbar werden. Bei geringerer Anzahl werden diese Lücken einfacher *“aufgesaugt”*.

AUFGABE 4: Zuletzt wollen wir uns noch anschauen, wie wir relative anstatt absolute Häufigkeiten angeben können. Nutzen Sie hierfür das `histnorm`-Argument und setzen es auf `"probability"`:

```
[12]: df["Strom / A"].plot(kind="histogram", nbins=100, histnorm="probability")
```

Natürlich ergibt es noch Sinn, die anderen Merkmale zu visualisieren. Da Ihnen die Funktionsaufrufe hierfür aber nun bekannt sein sollten, möchten wir an dieser Stelle darauf verzichten. Stattdessen wollen wir unsere Häufigkeitsverteilungen in einigen Kenngrößen zusammenzufassen. Dafür nutzen wir pandas' `describe()`-Funktion und setzen `include` auf `"all"`:

```
[13]: df.describe(include="all")
```

```
[13]:
```

	Maschine	Mode	Produkt	Strom / A	Drehmoment / Nm	Drehzahl / 1/min	\
count	1000	999.0	1000	999.000000	1000.000000	1000.000000	
unique	3	3.0	3	NaN	NaN	NaN	
top	C	2.0	Y	NaN	NaN	NaN	
freq	416	518.0	541	NaN	NaN	NaN	
mean	NaN	NaN	NaN	22.501915	48.555380	1460.945000	
std	NaN	NaN	NaN	2.520593	3.692908	1.916154	
min	NaN	NaN	NaN	18.221000	41.430000	1456.000000	
25%	NaN	NaN	NaN	20.435500	45.527500	1460.000000	
50%	NaN	NaN	NaN	22.102000	47.515000	1461.000000	
75%	NaN	NaN	NaN	24.668000	51.602500	1462.000000	
max	NaN	NaN	NaN	29.530000	60.110000	1467.000000	

	Temp Umgebung / degC	Temp Umrichter / degC	Temp Werkzeug / degC	\
count	1000.000000	1000.000000	1000.000000	
unique	NaN	NaN	NaN	
top	NaN	NaN	NaN	
freq	NaN	NaN	NaN	
mean	20.984700	23.750500	95.196400	

std	1.374695	1.040876	5.688506
min	17.300000	20.300000	82.900000
25%	20.100000	23.075000	90.600000
50%	21.000000	23.700000	94.000000
75%	21.900000	24.400000	99.800000
max	26.000000	27.400000	113.100000

	Bearbeitungszeit / s
count	1000.000000
unique	NaN
top	NaN
freq	NaN
mean	23.008900
std	1.970897
min	19.900000
25%	21.400000
50%	22.400000
75%	25.300000
max	25.300000

Durch diesen Funktionsaufruf werden eine ganze Menge Kenngrößen ausgegeben. Unter **count** wird die Anzahl von nicht-fehlenden Werten angegeben (also von Werten, die nicht “not a number” bzw. **NaN** sind), unter **unique** wird die Anzahl der Kategorien bei nominalen und ordinalen Daten angegeben, unter **top** der Modus und unter **freq** die Häufigkeit des Modus. Während uns **count** einen Aufschluss über die Datenqualität gibt, liefern uns **unique**, **top** und **freq** eigentlich keine zusätzliche Information im Vergleich zu unserer interaktiven Visualisierung. Interessanter ist die Angabe des arithmetischen Mittels **mean**, der Standardabweichung **std**, des Minimums **min**, des unteren Quartils **25%**, des mittleres Quartils bzw. Medians **50%**, des oberen Quartils **75%** sowie des Maximums **max**. Weitere Streuungsmaße wie die Varianz, die Spannweite oder der Interquartilsabstand können bei Bedarf aus diesen Kenngrößen einfach berechnet werden. Sie erinnern sich, dass jedes dieser Lage- und Streuungsmaße bestimmte Vor- und Nachteile aufweist. Eine allgemeine Empfehlung, welches Maß zu bevorzugen ist, kann man daher nicht abgeben.

AUFGABE 5: Wählen Sie nun ein metrisches Merkmal aus, bei dem es einen Unterschied 1 zwischen dem arithmetischen Mittel und dem Median gibt. Überlegen Sie sich, was Gründe für den Unterschied sein könnten. Überprüfen Sie Ihre Vermutungen dann durch eine passende Visualisierung!

Geben Sie nachfolgend Ihre schriftliche Antwort zur Aufgabe 5:

Ich vermute, dass der Unterschied zwischen **arithmetischem Mittel (ca. 48.56 Nm)** und **Median (ca. 47.52 Nm)** daher kommt, dass es einige wenige “Ausreißer” mit besonders hohen Drehmoments-Werten in den Daten gibt, die den Durchschnitt stark beeinflussen.

Im untenstehenden Diagramm sieht man eindeutig zwei Häufungen an Werten, die rechts und links des Medians und Durchschnitts liegen. Diese Häufungen scheinen *Normalverteilungen* um ca. 46 Nm (links) und 52 Nm (rechts) abzubilden. Eine dritte, in der Anzahl (also Höhe der Balken) deutlich kleinere Häufung befindet sich um ca. 57.5 Nm.

Diese ist vermutlich für den Unterschied zwischen Median und arithmetischem Mittel verant-

wortlich, da Werte, die eine große Abweichung vom Durchschnitt aufweisen, diesen stärker beeinflussen, als Werte, die nur eine geringe Abweichung aufweisen.

```
[14]: df["Drehmoment / Nm"].plot(kind="histogram")
```

1.2 Korrelationen untersuchen

Wir hoffen, dass wir Sie bereits von den Fähigkeiten von pandas und Plotly für die explorative Datenanalyse überzeugen konnten. Sie haben gesehen, dass es für die Darstellung eines Säulen- oder Balkendiagramms oder eines Histogramms jeweils nur einer einzigen Zeile Code bedarf; auch Lage- und Streuungsmaße können mit nur einer einzigen Zeile Code ausgegeben werden (bzw. drei Zeilen Code, wenn man auch unsere Implementierung für ordinale Merkmale verwenden möchte). Richtig interessant wird es oft aber erst, wenn wir nach weiteren Mustern und Zusammenhängen in den Daten suchen. Dazu müssen wir uns mehr als nur ein Merkmal zur gleichen Zeit anschauen. Wir gehen im Folgenden davon aus, dass wir nach unserer explorativen Analyse noch eine gezielte Analyse zu interessanten Mustern und Zusammenhängen planen. Um Aussagen über unsere Stichprobe hinaus treffen zu können, müssen wir unsere Erkenntnisse später auf einem unabhängigen Datensatz testen können und daher an dieser Stelle unsere Daten in einen Explorations- und einen Testdatensatz aufteilen. Wir nutzen dazu eine einfache Randomisierung. Wir wählen also mit der `sample()`-Funktion einen bestimmten Bruchteil unserer Daten zufällig zum Explorieren aus, zum Beispiel 75% bzw. 0.75. Um eine reproduzierbare Aufteilung zu erhalten, fixieren wir zudem den [Startwert des Zufallszahlengenerators](#), zum Beispiel auf 42:

```
[15]: import pandas as pd

df = pd.read_csv("data/machine_data.csv", delimiter=";", decimal=",")
df["Maschine"] = pd.Categorical(df["Maschine"])
df["Produkt"] = pd.Categorical(df["Produkt"])
df["Mode"] = pd.Categorical(df["Mode"], ordered=True)

exploration_data = df.sample(frac=0.75, random_state=42)
exploration_data
```

```
[15]:
```

	Maschine	Mode	Produkt	Strom / A	Drehmoment / Nm	Drehzahl / 1/min	\
521	A	1.0	Y	19.328	43.92	1461	
737	C	1.0	Y	21.265	43.37	1459	
740	A	1.0	Y	20.198	45.96	1459	
660	C	3.0	X	25.700	52.28	1462	
411	C	1.0	Y	21.949	44.61	1464	
..	...	...	...	...	...	...	
641	A	2.0	Y	22.299	50.61	1463	
915	C	2.0	X	22.801	46.49	1459	
752	C	3.0	X	26.118	53.16	1462	
806	A	1.0	Y	20.283	46.11	1461	
919	B	2.0	X	20.514	45.86	1461	

	Temp Umgebung / degC	Temp Umrichter / degC	Temp Werkzeug / degC	\
521	22.6	23.5	88.6	
737	19.8	23.0	86.7	
740	19.2	23.2	90.9	
660	22.8	25.5	102.4	
411	18.9	22.5	87.9	
..	...	...	...	
641	19.8	22.6	97.6	
915	20.6	23.2	91.9	
752	20.4	22.7	101.1	
806	19.3	23.3	91.0	
919	21.5	24.2	91.9	

	Bearbeitungszeit / s
521	25.3
737	25.3
740	25.3
660	19.9
411	25.3
..	...
641	23.8
915	21.4
752	19.9
806	25.3
919	22.4

[750 rows x 10 columns]

1.2.1 Zusammenhang von Merkmalen

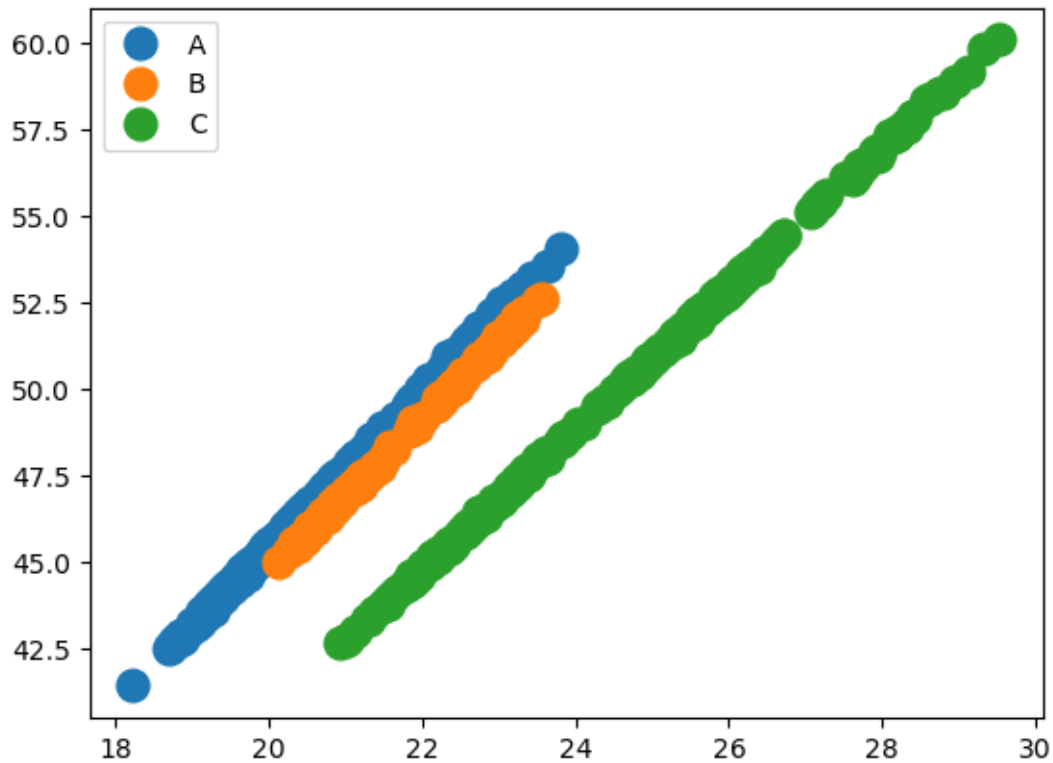
Nachdem wir nun einen Explorationsdatensatz erstellt haben, können wir uns den Zusammenhang von Merkmalen für diesen Datensatz anschauen. Hierzu erstellen wir ein [Streudiagramm](#), zum Beispiel für die Stromaufnahme in Ampere und das Drehmoment in Newtonmeter:

```
[16]: exploration_data.plot(kind="scatter", x="Strom / A", y="Drehmoment / Nm")
```

Für die Maschinen müssen wir den Code etwas umständlicher formulieren, da sich die Plot-Funktion mit kategorischen Variablen etwas schwer tun kann...

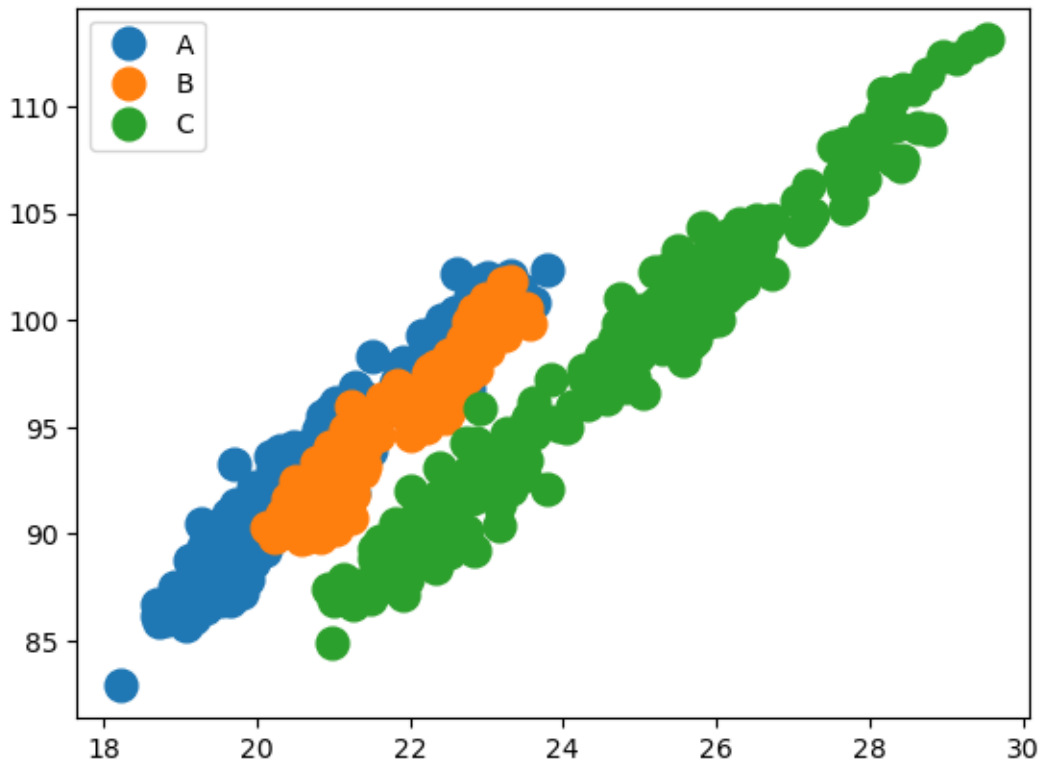
```
[17]: import matplotlib.pyplot as plt
groups = exploration_data.groupby('Maschine')

fig, ax = plt.subplots()
for name, group in groups:
    ax.plot(group['Strom / A'], group['Drehmoment / Nm'], marker='o',
            linestyle='', ms=12, label=name)
_ = ax.legend()
```

AUFGABE 6: Die beiden Streudiagramme zeigen, dass sich die drei geraden Linien durch die verschiedenen Maschinen ergeben. Testen Sie als nächstes den Zusammenhang zwischen der Stromaufnahme in Ampere und der Werkzeug-Temperatur in °Celsius. Was können Sie erkennen?

```
[18]: fig, ax = plt.subplots()
      for name, group in groups:
          ax.plot(group['Strom / A'], group['Temp Werkzeug / degC'], marker='o',
                  linestyle='', ms=12, label=name)
      _ = ax.legend()
```



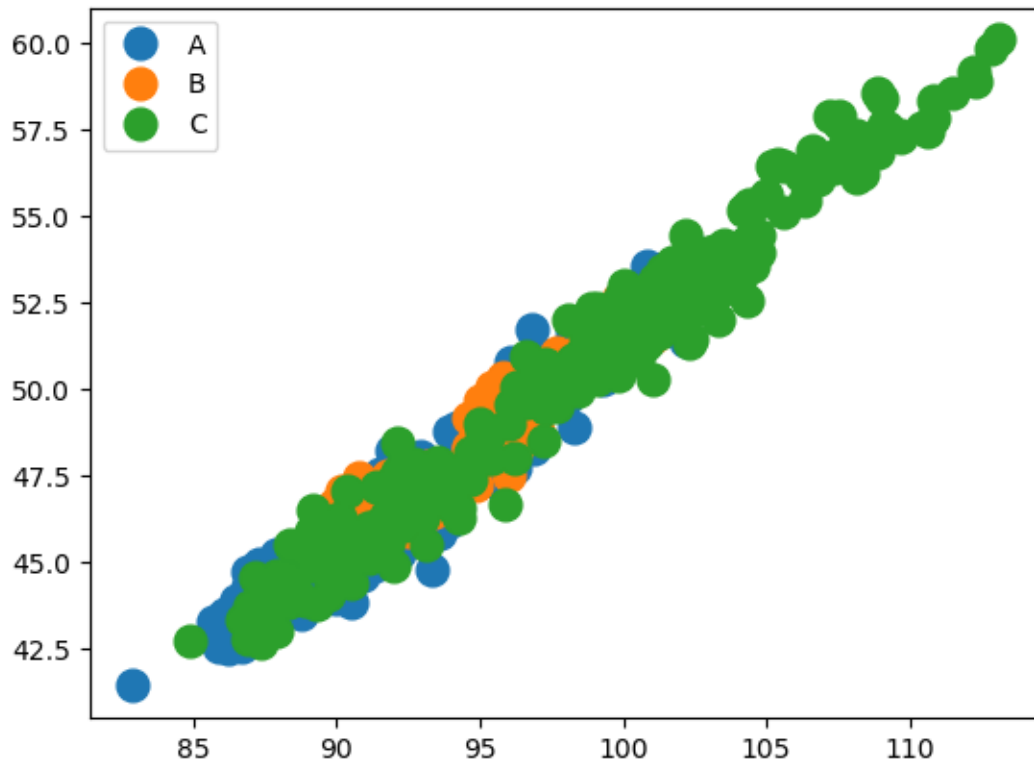
Geben Sie nachfolgend Ihre schriftliche Antwort zur AUFGABE 6:

Wie bereits in dem vorigen Diagramm zeigt sich hier eine klare lineare Abhängigkeit zwischen den verglichenen Messwerten für jede der drei Maschinen. Unterschiedlich sind hier allerdings die Abweichungen der Punkte vom “idealen” linearen Verlauf. Diese Abweichungen sind hier viel größer als im obigen, wo sie nahezu perfekte Linien bilden.

Es gibt also deutlich mehr Streuung, doch ist die Abhängigkeit vom genutzten Strom und der Temperatur des Werkzeugs deutlich zu sehen.

Wenn Werkzeugtemperatur vom Strom und das Drehmoment ebenfalls vom Strom abhängig ist, dann sollte sich auch eine interessante Abhängigkeit zwischen Temperatur und Drehmoment zeigen:

```
[19]: fig, ax = plt.subplots()
      for name, group in groups:
          ax.plot(group['Temp Werkzeug / degC'], group['Drehmoment / Nm'],
                  marker='o', linestyle='', ms=12, label=name)
      _ = ax.legend()
```



Auch hier zeigt sich deutlich die lineare Abhängigkeit, allerdings liegen die Werte der drei Maschinen nun übereinander.

AUFGABE 6.1: Um Ihnen eine bessere Intuition über die Größe und Richtung eines Zusammenhangs zu geben, haben wir eine Auswahl an Streudiagrammen mit unterschiedlichen Zusammenhängen zusammengestellt. Versuchen Sie, die unten aufgeführten Zusammenhänge einzuschätzen. Ist der folgende Zusammenhang groß oder klein, positiv oder negativ?

Geben Sie nachfolgend Ihre schriftliche Antwort zur AUFGABE 6.1:

Die Linie zeigt einen starken positiven linearen Zusammenhang zwischen der X- und Y-Achse, mit einem Faktor gegen 1.

AUFGABE 6.2: Wie sieht es bei dem nächsten Zusammenhang aus?

Geben Sie nachfolgend Ihre schriftliche Antwort zur AUFGABE 6.2:

Diese Linie zeigt einen starken negativen linearen Zusammenhang mit einem Faktor gegen -1.

AUFGABE 6.3: Überlegen Sie, wie ein Streudiagramm aussehen würde, wenn es keinen Zusammenhang zwischen den Merkmalen gäbe. Beschreiben Sie dann den folgenden Zusammenhang:

Geben Sie nachfolgend Ihre schriftliche Antwort zur AUFGABE 6.3:

Ein Diagramm ohne Zusammenhang der Achsen wäre ein völlig wirrer, ungeordneter Haufen Punkte. Hier zeigt sich jedoch etwas, das einer Parabel ähnelt. Dieser etwas schwache, aber definitiv bestehende Zusammenhang ist durch recht große Streuung eher als ein Band anstatt einer Linie zu erkennen.

AUFGABE 6.4: Schätzen Sie zum Schluss noch diesen Zusammenhang ein:

Geben Sie nachfolgend Ihre schriftliche Antwort zur AUFGABE 6.4:

Hier zeigt sich ein ähnlicher Zusammenhang wie in **[AUFGABE 6.1]**, allerdings mit deutlich mehr Streuung. Das heißt, dass dieser Zusammenhang etwas schwächer ist als der erste.

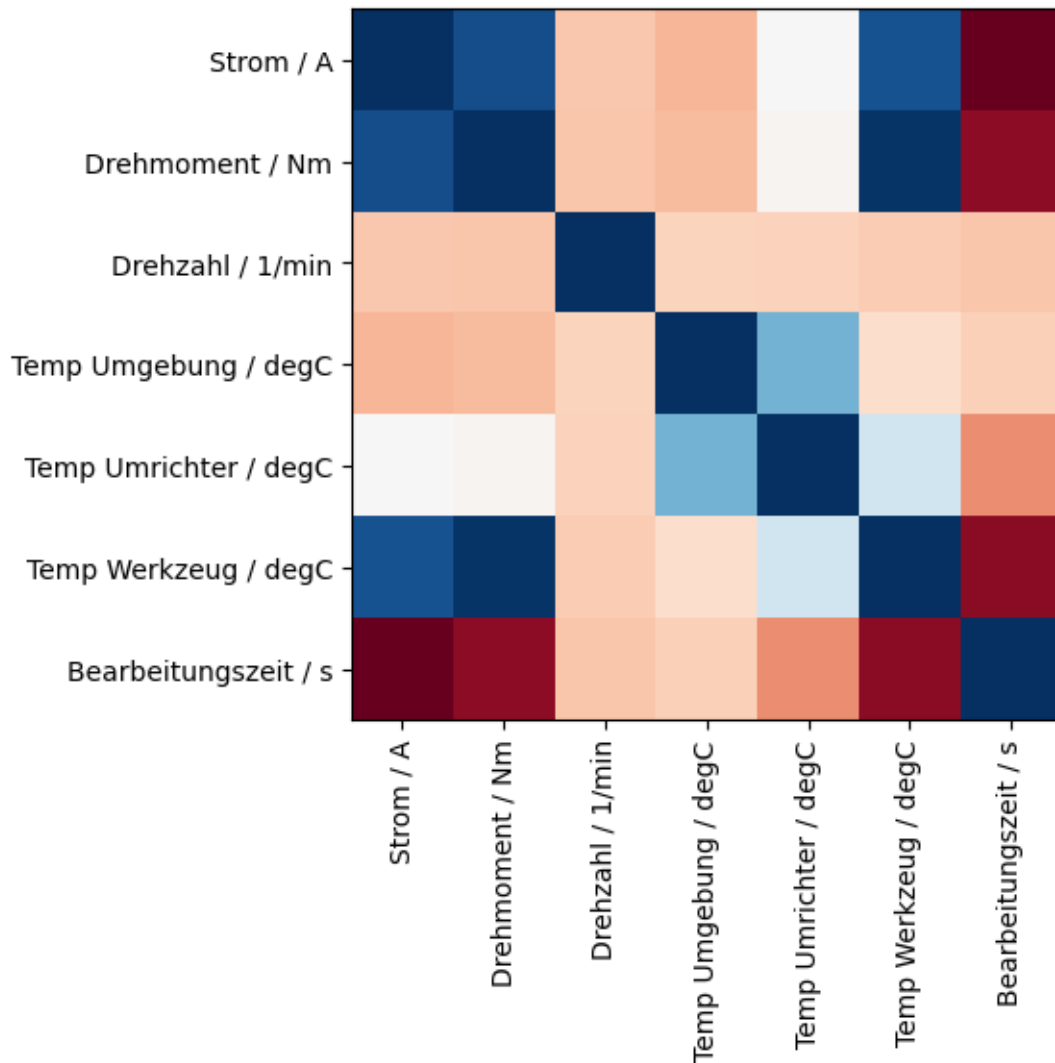
Würde man Trendlinien für beide Diagramme einzeichnen, lägen diese vermutlich sehr nah aneinander.

Möchten Sie Ihre Intuition über die Größe von Zusammenhängen (mit dem Korrelationskoeffizienten nach Pearson) noch weiter ausbauen, empfehlen wir Ihnen das Online-Spiel [Guess the Correlation](#).

1.3 Korrelationen und Transformationen

Mit dem besseren Verständnis über die Stärke und Richtung von Zusammenhängen können wir uns nun direkt die Korrelationskoeffizienten nach Pearson für unsere metrischen Merkmale anschauen. Dazu benutzen wir die `corr()`-Funktion (zur Bestimmung der Korrelationen) in Verbindung mit `"plt.imshow"` (kurz für *image show*) und passenden Farben (`"RdBu_r"`) zur Erstellung einer [Heatmap](#). Versuchen Sie, sich anhand der Farben und Ihrem Wissen über den Pearsons Correlation einen Reim aus der Matrix zu machen:

```
[20]: corr = exploration_data.corr(numeric_only=True)
      plt.imshow(corr, cmap="RdBu_r")
      plt.xticks(range(len(corr)), corr.columns, rotation=90)
      _ = plt.yticks(range(len(corr)), corr.index)
```



Die Matrix ist etwas schwer zu lesen und es ist mühsam, die Label ordentlich an die Achsen zu schreiben. Dankenswerterweise gibt es unzählige Möglichkeiten, mit **pandas** und den zugehörigen Visualisierungsbibliotheken wie z.B. **plotly**, **matplotlib** und **seaborn** ansprechende Visualisierungen zu gestalten. Die Kunst liegt vor allem darin, die Bibliotheken auseinander zu halten und die richtige auszuwählen, z.B. um eine geclusterte Heatmap zu erzeugen:

```
[21]: corr.style.background_gradient(vmin=-1, vmax=1, cmap="coolwarm")
```

```
[21]: <pandas.io.formats.style.Styler at 0x17428b0e0>
```

Deckt sich die berechnete Korrelation mit Ihren Erkenntnissen aus den Streudiagrammen? Fallen Ihnen noch weitere interessante Zusammenhänge auf? Manchmal ergeben sich interessante Zusammenhänge erst, wenn man verschiedene Merkmale miteinander kombiniert. So können wir zum Beispiel die Spannung aus der Drehzahl, dem Drehmoment und der Stromstärke [berechnen](#). Lassen Sie uns hierzu eine neue Spalte in unserer Tabelle anlegen:

```
[22]: exploration_data["Spannung / V"] = (2 * 3.14 / 60 * exploration_data["Drehzahl /  
↪ 1/min"] *  
exploration_data["Drehmoment / Nm"] / exploration_data["Strom / A"])
```

Nun können wir die Korrelationskoeffizienten nach Pearson für unser neues Merkmal noch einmal darstellen:

```
[23]: corr = exploration_data.corr(numeric_only=True)  
corr.style.background_gradient(vmin=-1, vmax=1, cmap="coolwarm")
```

```
[23]: <pandas.io.formats.style.Styler at 0x174169450>
```

Ergeben sich durch unser neues Merkmal neue interessante Zusammenhänge? Versuchen Sie nun selbst, eine Hypothese darüber aufzustellen, welche Merkmalskombination interessante Zusammenhänge zeigen könnte. Ohne Wissen über mechatronische Systeme erwarten wir natürlich nicht, dass Ihre Hypothese plausibel ist. Für unsere Zwecke würde daher auch die Hypothese genügen, dass sich durch die Multiplikation oder Division zweier beliebiger Merkmale möglicherweise ein neues interessantes Merkmal ergeben könnte.

AUFGABE 7: Legen Sie in der Tabelle dann eine neue Spalte für Ihr Merkmal an und lassen Sie sich die Korrelationskoeffizienten nach Pearson in einer Heatmap ausgeben. Nicht-benötigte Spalten können Sie bei Bedarf mit `del exploration_data["..."]` löschen, indem sie ... durch den Spaltennamen ersetzen. Achtung - solche Befehle können Sie offensichtlich nicht wiederholt ausfüllen.

```
[24]: exploration_data["M*n Leistung / W"] = 2 * 3.14 * exploration_data["Drehzahl /  
↪ 1/min"] * exploration_data["Drehmoment / Nm"]  
exploration_data["U*I Leistung / W"] = exploration_data["Strom / A"] *  
↪ exploration_data["Spannung / V"]  
  
corr = exploration_data.corr(numeric_only=True, method="spearman")  
corr.style.background_gradient(vmin=-1, vmax=1, cmap="coolwarm")
```

```
[24]: <pandas.io.formats.style.Styler at 0x174277ed0>
```

```
[25]: corr = exploration_data.corr(numeric_only=True, method="kendall")  
corr.style.background_gradient(vmin=-1, vmax=1, cmap="coolwarm")
```

```
[25]: <pandas.io.formats.style.Styler at 0x1791c25d0>
```

```
[26]: corr = exploration_data.corr(numeric_only=True, method="pearson")  
corr.style.background_gradient(vmin=-1, vmax=1, cmap="coolwarm")
```

```
[26]: <pandas.io.formats.style.Styler at 0x1791c2e90>
```

Antwort zur AUFGABE 7:

Die auf zwei Wege berechnete Leistung zeigt einen klaren Zusammenhang zwischen Strom, Drehzahl und (überraschenderweise) Werkzeugtemperatur! Da die Spannung, die zum berechnen der 2. "U*I

Leistung"-Spalte genutzt wurde, zuvor mithilfe der Gleichung der ersten "M*n Leistung"-Spalte bestimmt wurde, sind die beiden Spalten natürlich stark korreliert.

Prima! Sie haben mit dieser Übung einiges erreicht. Sie können nun sowohl Säulen- oder Balkendiagramme, Histogramme, Streudiagramme als auch Heatmaps in Plotly visualisieren, um einen Datensatz zu erkunden - alles in jeweils nur einer einzigen Zeile Code. Sie haben zudem die wichtigsten Funktionsargumente kennengelernt und haben einige davon auch selbst angepasst. Dass Sie auch eigene Hypothesen aufstellen und testen können, eröffnet Ihnen die Möglichkeit, Ihre eigenen Annahmen auch ohne eine gezielte Datenanalyse auf die Probe zu stellen.